

gRPC Client–Server Communication Flow

How Spring Boot services communicate through gRPC using a shared Protobuf contract

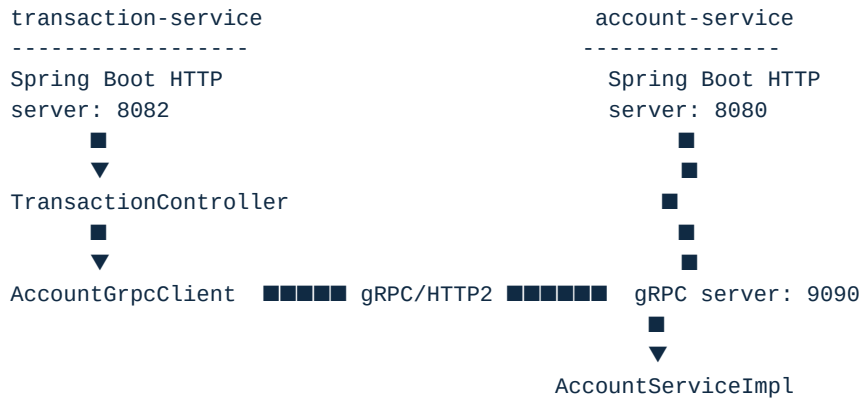
Core idea

The client and server use the same generated contract classes. Spring Boot starts the application, while the gRPC Spring Boot integration starts the gRPC server on its configured port.

Learning reference

1. The Communication Architecture

transaction-service exposes an HTTP endpoint and acts as the gRPC client. account-service exposes its normal Spring Boot HTTP server and also runs a gRPC server.



The client does not directly invoke a Java method inside the other application. It sends a network request to the gRPC server through the generated client stub.

2. Start Spring Boot and gRPC Together

The account service uses Spring Boot to run both HTTP and gRPC communication in the same application. The gRPC implementation is registered as a Spring bean using the `@GrpcService` annotation.

```
@GrpcService
public class AccountServiceImpl
    extends AccountServiceGrpc.AccountServiceImplBase {

    // Implement the RPC method here
}
```

The application configuration defines separate ports for HTTP and gRPC:

```
spring:
  application:
    name: account-service

server:
  port: 8080

grpc:
  server:
    port: 9090
```

Starting `AccountServiceApplication` starts the Spring Boot application, the HTTP server on port 8080, and the gRPC server on port 9090. A separate standalone `GrpcServer` class is not required.

3. Create the gRPC Client Channel and Stub

The transaction service creates a channel pointing to the account service's gRPC port, which is 9090 in this example.

```
ManagedChannel channel = ManagedChannelBuilder
    .forAddress("localhost", 9090)
```

```

        .usePlaintext()
        .build();

AccountServiceGrpc.AccountServiceBlockingStub stub =
    AccountServiceGrpc.newBlockingStub(channel);

```

The generated blocking stub provides Java methods that represent the remote RPC methods defined in the Protobuf contract.

4. Build and Send the Request

The client creates a request using the generated Protobuf builder and sends it through the stub.

```

GetAccountBalanceRequest request =
    GetAccountBalanceRequest.newBuilder()
        .setAccountId("ACC-1001")
        .build();

GetAccountBalanceResponse response =
    stub.getAccountBalance(request);

```

5. Server Receives and Processes the Request

The gRPC framework routes the incoming request to the method implemented in the class annotated with `@GrpcService`.

```

@Override
public void getAccountBalance(
    GetAccountBalanceRequest request,
    StreamObserver<GetAccountBalanceResponse> responseObserver) {

    String accountId = request.getAccountId();

    GetAccountBalanceResponse response =
        GetAccountBalanceResponse.newBuilder()
            .setAccountId(accountId)
            .setBalanceInPaisa(125000)
            .setCurrency("NPR")
            .build();

    responseObserver.onNext(response);
    responseObserver.onCompleted();
}

```

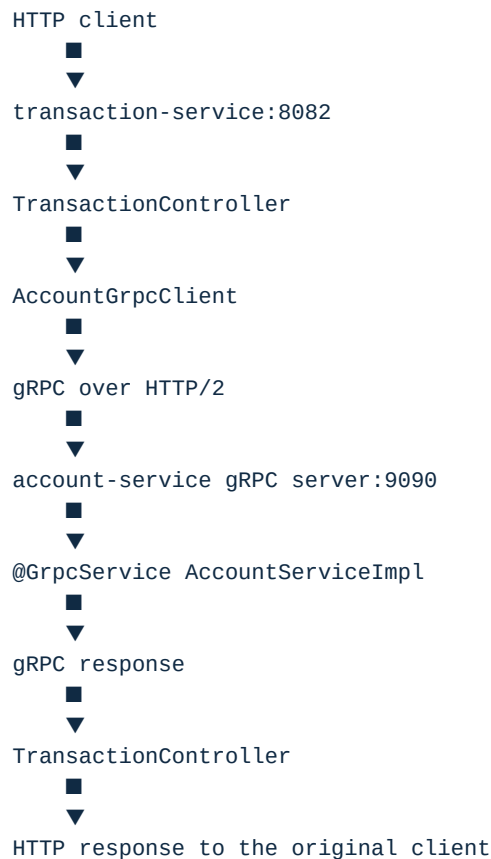
6. Return the Response

The server sends the response through the `StreamObserver`. The client receives the response from the blocking stub call.

Server sends	Client receives
Account ID: ACC-1001 Balance: 125000 Currency: NPR	A generated GetAccountBalanceResponse object containing the returned values.

7. Complete HTTP-to-gRPC-to-HTTP Flow

When a REST endpoint in transaction-service triggers the gRPC call, the complete flow is:



8. Complete Summary

Step	What happens
1	AccountServiceApplication starts Spring Boot HTTP on port 8080 and gRPC on port 9090.
2	The @GrpcService annotation registers the server implementation.
3	The transaction service creates a channel and client stub targeting port 9090.
4	The client builds a Protobuf request.
5	The stub sends the request to the remote gRPC server.
6	The annotated server implementation processes the request.
7	The server sends a Protobuf response, which the client receives and uses.

Key takeaway: One Spring Boot application can expose both HTTP and gRPC endpoints. The HTTP server uses `server.port`, while the gRPC server uses `grpc.server.port`. The `@GrpcService` annotation registers the gRPC implementation with Spring.